

TECH

MICROFRONTENDS / MODULE FED

MICROFRONTENDS / MONOREPO

MICROFRONTENDS / MONOREPO

35 ВОПРОСОВ 7 ДНЕЙ КОНСПЕКТ ЛЕКЦИЙ

ИСТОЧНИК module-federation.io / nx / общая практика

20 ВОПРОСОВ 7 ДНЕЙ

ДЕНЬ 1 СРЕДНИЙ	MFE Q1 · Q2 · Q3 · Q4	КОГДА OVERENGINEERING	MF	MFE	КОГДА	OVERENGINEERING	MF	
ДЕНЬ 2 ТЯЖЁЛЫЙ	HOST Q5 · Q6 · Q7 · Q8 · Q9	REMOTE VERSION SKEW	SHARED	РИСКИ V	HOST Q5 · Q6 · Q7 · Q8 · Q9	REMOTE VERSION SKEW	SHARED	РИСКИ V
ДЕНЬ 3 ТЯЖЁЛЫЙ	REACT VERSION Q10 · Q11 · Q12 · Q13	RUNTIME COMPAT	INDEPENDENT DEPLOY CONTRACTS		REACT VERSION Q10 · Q11 · Q12 · Q13	RUNTIME COMPAT	INDEPENDENT DEPLOY CONTRACTS	
ДЕНЬ 4 СРЕДНИЙ	SHARE TYPES Q14 · Q15 · Q16 · Q17	DS	AUTH	ROUTING	SHARE TYPES Q14 · Q15 · Q16 · Q17	DS	AUTH	ROUTING
ДЕНЬ 5 ТЯЖЁЛЫЙ	ERROR ISOLATION Q18 · Q19 · Q20 · Q21	ROLLBACK	FALLBACK	MONITOR	ERROR ISOLATION Q18 · Q19 · Q20 · Q21	ROLLBACK	FALLBACK	MONITOR
ДЕНЬ 6 СРЕДНИЙ	TESTING Q22 · Q23 · Q24 · Q25 · Q26 · Q27	LOCAL DEV MIGRATION	UX CONSISTENCY STRANGLER		TESTING Q22 · Q23 · Q24 · Q25 · Q26 · Q27	LOCAL DEV MIGRATION	UX CONSISTENCY STRANGLER	
ДЕНЬ 7 СРЕДНИЙ	MONOREPO vs MF Q28 · Q29 · Q30 · Q31 · Q32 · Q33 · Q34 · Q35	ПАКЕТЫ PACKAGES	КОГДА MONOREPO SHARED CONFIG	ПИ AF	MONOREPO vs MF Q28 · Q29 · Q30 · Q31 · Q32 · Q33 · Q34 · Q35	ПАКЕТЫ PACKAGES	КОГДА MONOREPO SHARED CONFIG	ПИ AF

ДЕНЬ 1 НАГРУЗКА СРЕДНИЙ 4 ВОПРОСА

MFE КОГДА OVERENGINEERING MF

ЧТО УЧИМ В ЭТОТ ДЕНЬ

MFE КОГДА OVERENGINEERING MF

ВОПРОСЫ ДНЯ

- Q01 Что такое microfrontend?
- Q02 Когда microfrontends оправданы?
- Q03 Когда microfrontends являются overengineering?
- Q04 Что такое Module Federation?

Что такое microfrontend?

ОПИСАНИЕ

Архитектурный подход: разбить большое frontend-приложение на независимо разрабатываемые/деплойщиеся части. Аналог микросервисов на frontend. Несколько команд работают параллельно.

КАК РАБОТАЕТ

- 01 Independent dev / deploy.
- 02 Multiple teams.
- 03 Composition в shell.
- 04 Tech-stack flexibility.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Multi-team большие orgs.
- 02 Long-lived enterprise.

ПОДВОДНЫЕ КАМНИ

- 01 Overhead для small team.
- 02 Coordination shared deps.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое shell?
- 02 Чем MFE от modules отличается?

ПОЛНАЯ СТАТЬЯ

<https://martinfowler.com/articles/micro-frontends.html>

ВОПРОС Q02

Когда microfrontends оправданы?

ОПИСАНИЕ

Когда: 5+ команд работают над одним приложением, разные tech stacks, независимая частота релизов, разные SLA, scale-issues, organizational coupling нужно разорвать.

КАК РАБОТАЕТ

- 01 5+ teams.
- 02 Different tech stacks.
- 03 Independent releases.
- 04 Different SLAs.
- 05 Conway's law.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Enterprise apps.
- 02 Multi-product platforms.

ПОДВОДНЫЕ КАМНИ

- 01 Без real coordination need – overkill.
- 02 Single team adopting MF – pain.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое Conway's law?
- 02 Когда modular monolith достаточен?

ПОЛНАЯ СТАТЬЯ

<https://martinfowler.com/articles/micro-frontends.html>

Когда microfrontends являются overengineering?

ОПИСАНИЕ

Single team / small app / однообразный stack / coordinated releases / shared infra. MF добавляет complexity (build, deploy, runtime, debugging) – без real benefit оправданы редко.

КАК РАБОТАЕТ

- 01 Single team.
- 02 Small app.
- 03 Same stack.
- 04 Coordinated releases.
- 05 No deploy independence need.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Architecture decisions.
- 02 Adoption review.

ПОДВОДНЫЕ КАМНИ

- 01 MF 'because trendy' – мощный source of pain.
- 02 Migration without need wastes months.

МОГУТ ДОСПРОСИТЬ

- 01 Когда modular monolith?
- 02 Чем complexity overhead?

ПОЛНАЯ СТАТЬЯ

<https://martinfowler.com/articles/micro-frontends.html#TheDownsides>

ВОПРОС Q04

Что такое Module Federation?

ОПИСАНИЕ

Webpack 5+ feature: runtime composition of multiple builds. Host загружает remote chunks dynamically. Shared deps (singleton). Один из самых популярных MF-инструментов.

КАК РАБОТАЕТ

- 01 Webpack 5 plugin.
- 02 Runtime composition.
- 03 Dynamic remote loading.
- 04 Shared singleton deps.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 MFE adoption.
- 02 Plugin systems.

ПОДВОДНЫЕ КАМНИ

- 01 Webpack-only (Vite plugin есть, но less mature).
- 02 Shared config complex.

МОГУТ ДОСПРОСИТЬ

- 01 Чем native MF от build-time integration отличается?
- 02 Что такое eager / lazy share?

ПОЛНАЯ СТАТЬЯ

<https://module-federation.io/>

HOST**REMOTE****SHARED****РИСКИ****VERSION SKEW**

ЧТО УЧИМ В ЭТОТ ДЕНЬ

HOST REMOTE SHARED РИСКИ VERSION SKEW

ВОПРОСЫ ДНЯ

Q05 Что такое host?

Q06 Что такое remote?

Q07 Что такое shared dependency?

Q08 Какие риски у shared dependencies?

Q09 Что такое version skew?

ВОПРОС Q05

Что такое host?

ОПИСАНИЕ

Application-shell, в которое монтируются remotes. Owns routing, auth, layout, shared services. Загружает remotes runtime через MF. Часто называется 'shell' или 'container'.

КАК РАБОТАЕТ

- 01 Application shell.
- 02 Loads remotes at runtime.
- 03 Owns routing/auth/layout.
- 04 Provides shared services.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 MFE setup.
- 02 Application platform.

ПОДВОДНЫЕ КАМНИ

- 01 Host bottleneck для releases.
- 02 Shell broken → all remotes down.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое app-shell?
- 02 Чем host от gateway отличается?

ПОЛНАЯ СТАТЬЯ

<https://module-federation.io/>

Что такое remote?

ОПИСАНИЕ

Independent build, который exports modules для consumption host'ом. Имеет свой repo / CI / deploy. Загружается host'ом runtime. Может работать standalone (для dev).

КАК РАБОТАЕТ

- 01 Independent build.
- 02 Exposes modules.
- 03 Own CI/deploy.
- 04 Standalone dev mode.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 MFE adoption.

ПОДВОДНЫЕ КАМНИ

- 01 Без contract – break при update.
- 02 Standalone-mode не отражает production-host.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое expose?
- 02 Чем module от entry-point отличается?

ПОЛНАЯ СТАТЬЯ

<https://module-federation.io/docs/getting-started/glossary>

ВОПРОС Q 07

Что такое shared dependency?

ОПИСАНИЕ

Зависимость, которую host и remotes используют ИЗ ОДНОГО bundle (singleton) – например, React. Без shared – N копий React → break хуков. Конфигурируется в MF-config.

КАК РАБОТАЕТ

- 01 shared: { react: { singleton: true } }.
- 02 Single instance shared.
- 03 Required для React, Redux.
- 04 Version-matched.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 MFE с React.
- 02 Shared state libraries.

ПОДВОДНЫЕ КАМНИ

- 01 Without singleton – broken hooks.
- 02 Version mismatch – runtime error.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое requiredVersion?
- 02 Чем eager от lazy share отличается?

ПОЛНАЯ СТАТЬЯ

<https://module-federation.io/configure/shared>

Какие риски у shared dependencies?

ОПИСАНИЕ

Version skew (разные мажорные → break), upgrade-coordination (все team должны обновить), bundle-bloat (eager-share включает в bundle), runtime-error при mismatch.

КАК РАБОТАЕТ

- 01 Version skew = broken.
- 02 Upgrade требует coordination.
- 03 Eager share = bundle bloat.
- 04 Runtime errors hard to debug.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 MF planning.
- 02 Upgrade strategies.

ПОДВОДНЫЕ КАМНИ

- 01 Без strict version constraints – silent skew.
- 02 Coordination overhead.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое requiredVersion?
- 02 Чем strictVersion полезен?

ПОЛНАЯ СТАТЬЯ

<https://module-federation.io/configure/shared>

ВОПРОС Q09

Что такое version skew?

ОПИСАНИЕ

Разные версии shared dep между host/remotes. React 18 в shell, React 17 в remote → cannot use hooks outside Component. Сложно дебажить.

КАК РАБОТАЕТ

- 01 Different shared dep versions.
- 02 Most painful с React.
- 03 Hooks errors.
- 04 Runtime crashes.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 MF debugging.
- 02 Multi-team coordination.

ПОДВОДНЫЕ КАМНИ

- 01 Loose constraints (^18) → mismatch silently.
- 02 Каждый team upgrade на своей timeline.

МОГУТ ДОСПРОСИТЬ

- 01 Чем strictVersion помогает?
- 02 Что такое version-locked monorepo?

ПОЛНАЯ СТАТЬЯ

<https://module-federation.io/blog/version-mismatch>

REACT VERSION**INDEPENDENT DEPLOY****RUNTIME COMPAT**

ЧТО УЧИМ В ЭТОТ ДЕНЬ

REACT VERSION

INDEPENDENT DEPLOY

RUNTIME COMPAT

CONTRACTS

ВОПРОСЫ ДНЯ

Q10 Как контролировать shared React version?

Q11 Как обеспечить independent deployment?

Q12 Как обеспечить runtime compatibility?

Q13 Как проектировать contracts между shell и remotes?

ВОПРОС Q10

Как контролировать shared React version?

ОПИСАНИЕ

shared: { react: { singleton: true, requiredVersion: '^18.2.0', strictVersion: true } }. CI-проверка version match. Coordination через release-train или synced upgrades.

КАК РАБОТАЕТ

- 01 singleton: true + strictVersion.
- 02 CI check на version match.
- 03 Coordinated upgrades.
- 04 Release-train approach.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 MF с React.
- 02 Любой shared lib.

ПОДВОДНЫЕ КАМНИ

- 01 Loose constraints → skew.
- 02 Strict без plan upgrade – block releases.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое release-train?
- 02 Чем locked-versions полезны?

ПОЛНАЯ СТАТЬЯ

<https://module-federation.io/configure/shared>

Как обеспечить independent deployment?

ОПИСАНИЕ

Каждый remote – own CI/CD. Versioned URLs (cdn.com/)). Host config с pin / latest. Versioning + rollback. Avoid synchronous deploy dependencies.

КАК РАБОТАЕТ

- 01 Per-remote CI/CD.
- 02 Versioned URL deployment.
- 03 Host pins remote-version.
- 04 Independent rollback.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 MFE setup.

ПОДВОДНЫЕ КАМНИ

- 01 Если versioned URLs нет – coupling.
- 02 Sync deploys = monolith.

МОГУТ ДОСПРОСИТЬ

- 01 Чем versioned URL полезен?
- 02 Как rollback?

ПОЛНАЯ СТАТЬЯ

<https://module-federation.io/>

ВОПРОС Q12

Как обеспечить runtime compatibility?

ОПИСАНИЕ

Versioned APIs между host и remotes. Contract testing. SemVer для remote-versions. Backwards-compatible изменения. Deprecation period перед removal.

КАК РАБОТАЕТ

- 01 Contract per remote.
- 02 SemVer versioning.
- 03 Backwards-compatible changes.
- 04 Deprecation period.
- 05 Contract tests in CI.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 MFE evolution.
- 02 Multi-team.

ПОДВОДНЫЕ КАМНИ

- 01 Breaking change без version bump → outage.
- 02 Без contract testing – рассинхрон.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое contract testing?
- 02 Чем SemVer полезен?

ПОЛНАЯ СТАТЬЯ

<https://docs.pact.io/>

Как проектировать contracts между shell и remotes?

ОПИСАНИЕ

Typed interface (TypeScript): что shell даёт remotes (auth, theme, navigate), что remotes exposes (mount, unmount, lifecycle). Versioned + documented + tested.

КАК РАБОТАЕТ

- 01 TS-interface for shell-remote contract.
- 02 Lifecycle: mount, unmount.
- 03 Shared services API.
- 04 Versioned + tested.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 MFE adoption.
- 02 Plugin platforms.

ПОДВОДНЫЕ КАМНИ

- 01 Без contract – каждое обновление рискованно.
- 02 TypeScript-interface не enforce runtime.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое module contract?
- 02 Чем .d.ts полезен?

ПОЛНАЯ СТАТЬЯ

<https://single-spa.js.org/docs/api>

ДЕНЬ 4

НАГРУЗКА

СРЕДНИЙ

4 ВОПРОСА

SHARE TYPES**DS****AUTH****ROUTING**

ЧТО УЧИМ В ЭТОТ ДЕНЬ

SHARE TYPES DS AUTH ROUTING

ВОПРОСЫ ДНЯ

Q14 Как шарить типы между remotes?

Q15 Как шарить design system?

Q16 Как шарить auth context?

Q17 Как шарить routing?

Как шарить типы между remotes?

ОПИСАНИЕ

Monorepo с shared types-package. Или published @company/shared-types npm. Зареферч types для federated modules через @module-federation/typescript. Generated d.ts файлы.

КАК РАБОТАЕТ

- 01 Monorepo types-package.
- 02 Published @company/types.
- 03 Federation types plugin.
- 04 Generated d.ts.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 MF + monorepo.
- 02 Multi-team typed apps.

ПОДВОДНЫЕ КАМНИ

- 01 Without sync – types out-of-date.
- 02 Type-only sharing – нужны runtime contracts тоже.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое @mf-types?
- 02 Чем generated d.ts полезен?

ПОЛНАЯ СТАТЬЯ

<https://module-federation.io/blog/typescript-support>

ВОПРОС Q15

Как шарить design system?

ОПИСАНИЕ

Shared package (DS-library) как singleton MF-share. Или published npm. Tokens через CSS-vars (cross-bundle). Версионирование с deprecation periods.

КАК РАБОТАЕТ

- 01 DS as MF singleton.
- 02 CSS-vars cross-bundle.
- 03 SemVer + Changesets.
- 04 Storybook docs shared.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Multi-team apps.
- 02 MFE platforms.

ПОДВОДНЫЕ КАМНИ

- 01 DS version skew → visual mismatch.
- 02 CSS-vars naming clashes.

МОГУТ ДОСПРОСИТЬ

- 01 Чем CSS-vars cross-bundle полезны?
- 02 Как версионировать DS?

ПОЛНАЯ СТАТЬЯ

<https://www.designsystems.com/>

Как шарить auth context?

ОПИСАНИЕ

Auth-state в shell (host), exposed через shared-context или service. Remotes consume через hook. Token-refresh централизован. Logout broadcast всем remotes.

КАК РАБОТАЕТ

- 01 Auth-state в shell.
- 02 Shared context / service.
- 03 Centralized refresh.
- 04 Broadcast logout.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 MFE с auth.

ПОДВОДНЫЕ КАМНИ

- 01 Each remote с own auth = duplicate.
- 02 Token refresh race без singleton.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое context-bridge?
- 02 Чем singleton auth полезен?

ПОЛНАЯ СТАТЬЯ

https://owasp.org/www-project-cheat-sheets/cheatsheets/Authentication_Cheat_Sheet

ВОПРОС Q17

Как шарить routing?

ОПИСАНИЕ

Shell owns top-level routing. Remotes mount по path. Sub-routing внутри remote – local. Linking через shell-API. Conflicts решает shell.

КАК РАБОТАЕТ

- 01 Shell top-level routing.
- 02 Remotes mount per path.
- 03 Sub-routing local в remote.
- 04 Shell-API для navigation.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 MFE shell.

ПОДВОДНЫЕ КАМНИ

- 01 Conflicting routes между remotes.
- 02 Sub-router conflicts с shell.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое path-based composition?
- 02 Чем nested routes полезны?

ПОЛНАЯ СТАТЬЯ

<https://reactrouter.com/>

ERROR ISOLATION**FALLBACK****MONITORING****ROLLBACK**

ЧТО УЧИМ В ЭТОТ ДЕНЬ

ERROR ISOLATION

FALLBACK

MONITORING

ROLLBACK

ВОПРОСЫ ДНЯ

Q18 Как изолировать ошибки remote app?

Q19 Как сделать fallback, если remote не загрузился?

Q20 Как мониторить remote loading failures?

Q21 Как делать rollback remote app?

ВОПРОС Q18

Как изолировать ошибки remote app?

ОПИСАНИЕ

ErrorBoundary вокруг каждого remote-mount. Logs в shell observability. Show fallback UI. Не позволять remote-error ломать shell. Sentry-tag по remote.

КАК РАБОТАЕТ

- 01 ErrorBoundary per remote.
- 02 Sentry-tag remote-name.
- 03 Fallback UI: 'feature unavailable'.
- 04 Logs в shell observability.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 MFE production.

ПОДВОДНЫЕ КАМНИ

- 01 Без EB → один remote ломает shell.
- 02 Slow fallback UX.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое error isolation?
- 02 Чем react-error-boundary помогает?

ПОЛНАЯ СТАТЬЯ

<https://github.com/bvaughn/react-error-boundary>

Как сделать fallback, если remote не загрузился?

ОПИСАНИЕ

Suspense fallback (загрузка). При error – ErrorBoundary с retry-кнопкой. После N retries – graceful 'feature unavailable'. Сообщить через observability.

КАК РАБОТАЕТ

- 01 Suspense + ErrorBoundary.
- 02 Retry mechanism.
- 03 Graceful 'unavailable' UI.
- 04 Alert observability.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 MF production.

ПОДВОДНЫЕ КАМНИ

- 01 Без fallback – broken shell.
- 02 Без alerting – silent failure.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое graceful degradation?
- 02 Как retry chunk-load?

ПОЛНАЯ СТАТЬЯ

<https://web.dev/articles/code-splitting-suspense>

ВОПРОС Q20

Как мониторить remote loading failures?

ОПИСАНИЕ

Track: chunk-load errors (window.error), retry counts, loading-times. Sentry / DataDog. Alert при rate growth. Connect к release-annotations.

КАК РАБОТАЕТ

- 01 Track chunk-load errors.
- 02 Histogram loading time.
- 03 Sentry tag per remote.
- 04 Alert на rate.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 MF production.

ПОДВОДНЫЕ КАМНИ

- 01 Default browser-error не пишется в monitoring.
- 02 Без tag – нельзя отделить per-remote.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое chunk-load error?
- 02 Чем release-annotation полезна?

ПОЛНАЯ СТАТЬЯ

<https://docs.sentry.io/>

Как делать rollback remote app?

ОПИСАНИЕ

Versioned URLs позволяют pin старую version в shell-config. Feature-flag для switch версий. CDN-cache aware. Auto-rollback при error spike.

КАК РАБОТАЕТ

- 01 Versioned remote URLs.
- 02 Shell config pin version.
- 03 Feature-flag switch.
- 04 Auto-rollback на error.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 MF production deploys.

ПОДВОДНЫЕ КАМНИ

- 01 Без versioning – нечего rollback.
- 02 CDN cache держит старое (purge needed).

МОГУТ ДОСПРОСИТЬ

- 01 Что такое immutable deploy?
- 02 Как auto-rollback?

ПОЛНАЯ СТАТЬЯ

<https://module-federation.io/>

ДЕНЬ 6

НАГРУЗКА

СРЕДНИЙ

6 ВОПРОСОВ

TESTING

LOCAL DEV

UX CONSISTENCY

MIGRATION

ST

ЧТО УЧИМ В ЭТОТ ДЕНЬ

TESTING

LOCAL DEV

UX CONSISTENCY

MIGRATION

STRANGLER

ВОПРОСЫ ДНЯ

Q22 Как тестировать federated modules?

Q23 Как делать local development с несколькими remotes?

Q24 Как обеспечить consistent UX между командами?

Q25 Как мигрировать SPA в microfrontends?

Q26 Что такое strangler pattern?

Q27 Как избежать big bang rewrite?

Как тестировать federated modules?

ОПИСАНИЕ

Unit-тесты внутри remote (как обычный код). Integration: mount remote в test-shell. Contract-тесты против shell-API. E2E через staging-shell с real remotes.

КАК РАБОТАЕТ

- 01 Unit как обычный код.
- 02 Integration в test-shell.
- 03 Contract tests.
- 04 E2E на staging.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 MF testing strategy.

ПОДВОДНЫЕ КАМНИ

- 01 Without contract – break post-deploy.
- 02 E2E с live remotes – flaky.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое contract testing?
- 02 Чем integration в test-shell полезен?

ПОЛНАЯ СТАТЬЯ

<https://docs.pact.io/>

В О П Р О С Q 2 3

Как делать local development с несколькими remotes?

ОПИСАНИЕ

Tools: @module-federation/enhanced, MF dev-tools. Run shell+remotes локально (parallel npm starts). Mock-mode: stub remotes для shell-only dev. Webpack-dev-server multi-target.

КАК РАБОТАЕТ

- 01 Multiple npm-start.
- 02 MF dev-tools.
- 03 Mock-mode для shell-only.
- 04 localhost ports per remote.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 MFE adoption.
- 02 Daily dev work.

ПОДВОДНЫЕ КАМНИ

- 01 Slow boot multiple processes.
- 02 Port conflicts.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое mock-mode?
- 02 Чем dev-tools помогают?

ПОЛНАЯ СТАТЬЯ

<https://module-federation.io/blog/mf-100>

Как обеспечить consistent UX между командами?

ОПИСАНИЕ

Shared design system (singleton). Governance / RFC process. Storybook for component-discovery. Visual regression tests. UX guidelines документированы.

КАК РАБОТАЕТ

- 01 Shared DS singleton.
- 02 Governance / RFC.
- 03 Visual regression in CI.
- 04 UX guidelines.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 MFE platforms.
- 02 Multi-team enterprise.

ПОДВОДНЫЕ КАМНИ

- 01 Каждый team со своим стилем – fragmentation.
- 02 DS в каждом – copies.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое RFC process?
- 02 Чем visual regression полезен?

ПОЛНАЯ СТАТЬЯ

<https://www.designsystems.com/>

В О П Р О С Q 2 5

Как мигрировать SPA в microfrontends?

ОПИСАНИЕ

Strangler-pattern: новые фичи как remotes, старый SPA – temporary host. Постепенно переезжать legacy в remotes. Shared services (auth, DS) первыми. Pilot – secondary feature.

КАК РАБОТАЕТ

- 01 Strangler-pattern.
- 02 Pilot – secondary feature.
- 03 Shared services первыми.
- 04 Gradual extract.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Brownfield SPA.
- 02 Adoption MF.

ПОДВОДНЫЕ КАМНИ

- 01 Big-bang rewrite – обычно убивает.
- 02 Without pilot – не учимся.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое strangler-pattern?
- 02 Какой первый remote выбрать?

ПОЛНАЯ СТАТЬЯ

<https://martinfowler.com/bliki/StranglerFigApplication.html>

Что такое strangler pattern?

ОПИСАНИЕ

Постепенная замена старой системы новой: новый код перехватывает требования, старый постепенно угасает. Названо в честь fig-vine, который душит дерево-host. Альтернатива big-bang rewrite.

КАК РАБОТАЕТ

- 01 New code intercepts gradually.
- 02 Old code shrinks over time.
- 03 No big-bang rewrite.
- 04 Continuous delivery возможна.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Legacy migrations.
- 02 Cloud-adoption.

ПОДВОДНЫЕ КАМНИ

- 01 Long migration period (years).
- 02 Two systems forever risk.

МОГУТ ДОСПРОСИТЬ

- 01 Чем от big-bang отличается?
- 02 Как считать прогресс?

ПОЛНАЯ СТАТЬЯ

<https://martinfowler.com/bliki/StranglerFigApplication.html>

ВОПРОС Q 27

Как избежать big bang rewrite?

ОПИСАНИЕ

Strangler-pattern. Pilot одна фича. Параллельная работа старого + нового. Метрики прогресса. Регулярная re-evaluation. Adoption metrics. Не ждать 'идеальной' новой архитектуры.

КАК РАБОТАЕТ

- 01 Strangler.
- 02 Pilot first.
- 03 Parallel old + new.
- 04 Metric tracking.
- 05 Iterative refinement.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Legacy modernization.
- 02 Architecture migrations.

ПОДВОДНЫЕ КАМНИ

- 01 Big-bang часто failing.
- 02 Без metric – не понимаем progress.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое pilot project?
- 02 Какие metrics?

ПОЛНАЯ СТАТЬЯ

<https://martinfowler.com/bliki/StranglerFigApplication.html>

MONOREPO vs MF

КОГДА MONOREPO

РИСКИ

PACKAGES

ЧТО УЧИМ В ЭТОТ ДЕНЬ

MONOREPO vs MF
VERSIONING

КОГДА MONOREPO
MODULAR MONOLITH

РИСКИ

PACKAGES

SHARED CONFIG

AFFECTED

ВОПРОСЫ ДНЯ

Q28 Чем monorepo отличается от microfrontends?

Q29 Когда monorepo полезен?

Q30 Какие риски у monorepo?

Q31 Как организовать packages в monorepo?

Q32 Как организовать shared lint/test/build conventions?

Q33 Как делать affected builds?

Q34 Как версионировать shared packages?

Q35 Когда лучше modular monolith, а не microfrontends?

ВОПРОС Q 2 8

Чем монорепо отличается от microfrontends?

ОПИСАНИЕ

Монорепо – все packages в одном репо, build-time integration. MF – runtime composition разных deployments. Можно совмещать (монорепо с MF), но это разные dimensions.

КАК РАБОТАЕТ

- 01 Монорепо: build-time, single repo.
- 02 MF: runtime, multiple deploys.
- 03 Совместимы (монорепо + MF).
- 04 Разные axes.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Architecture decision.
- 02 Adoption planning.

ПОДВОДНЫЕ КАМНИ

- 01 Confused – choose only one of two.
- 02 Монорепо не = single deploy.

МОГУТ ДОСПРОСИТЬ

- 01 Чем lerna от nx отличается?
- 02 Что такое polyrepo?

ПОЛНАЯ СТАТЬЯ

<https://nx.dev/concepts/more-concepts/why-monorepos>

Когда моногеро полезен?

ОПИСАНИЕ

Code-sharing между packages, atomic refactoring (cross-package), unified tooling, single source of truth для CI/CD/tests. Shared deps легко manage. Tooling: nx, turborepo, pnpm workspaces.

КАК РАБОТАЕТ

- 01 Cross-package code sharing.
- 02 Atomic cross-package PR.
- 03 Unified CI/CD.
- 04 Single tool config.
- 05 Tools: nx / turborepo / pnpm.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Multi-package frontend.
- 02 Component libs + apps.

ПОДВОДНЫЕ КАМНИ

- 01 Single repo growing huge.
- 02 Build slow без affected.
- 03 Permission issues (всё shared).

МОГУТ ДОСПРОСИТЬ

- 01 Что такое affected build?
- 02 Чем nx от turbo отличается?

ПОЛНАЯ СТАТЬЯ

<https://nx.dev/>

ВОПРОС Q30

Какие риски у моногеро?

ОПИСАНИЕ

Slow CI без incremental builds. Permission management (всё в одном repo). Coupling temptation (cross-package shortcuts). Big git-history. Tooling lock-in.

КАК РАБОТАЕТ

- 01 Slow CI.
- 02 Permission scope.
- 03 Coupling temptation.
- 04 Big history.
- 05 Tool lock-in.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Monorepo evaluation.

ПОДВОДНЫЕ КАМНИ

- 01 Без affected – каждый PR биллс всё.
- 02 Without boundaries – code coupling.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое affected detection?
- 02 Чем shallow clone полезен?

ПОЛНАЯ СТАТЬЯ

<https://nx.dev/concepts/more-concepts/dependency-graph>

Как организовать packages в монорепо?

ОПИСАНИЕ

Typical: apps/* (deployable apps), packages/* (libraries), tools/* (scripts).
Per-package package.json. Workspace manifest на root. Boundaries enforced через lint.

КАК РАБОТАЕТ

- 01 apps/ + packages/ + tools/.
- 02 Per-package package.json.
- 03 Root workspace manifest.
- 04 Boundaries via lint.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Frontend monorepos.

ПОДВОДНЫЕ КАМНИ

- 01 Flat structure → confusion.
- 02 Без boundaries – coupling.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое workspace?
- 02 Чем nx-tags полезны?

ПОЛНАЯ СТАТЬЯ

<https://nx.dev/concepts/more-concepts/library-types>

В О П Р О С Q 3 2

Как организовать shared lint/test/build conventions?

ОПИСАНИЕ

Root configs (.eslintrc, tsconfig, jest.config) с extends per-package. Shared scripts в /tools. nx targets / turborepo pipelines. Pre-commit hooks через husky / lefthook.

КАК РАБОТАЕТ

- 01 Root configs + per-package extends.
- 02 Shared scripts в /tools.
- 03 Nx targets / turbo pipeline.
- 04 Husky pre-commit.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Monorepos с many packages.

ПОДВОДНЫЕ КАМНИ

- 01 Per-package full configs – drift.
- 02 Без shared – duplicate.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое extends в tsconfig?
- 02 Чем nx generators полезны?

ПОЛНАЯ СТАТЬЯ

<https://nx.dev/concepts/more-concepts/library-types>

Как делать affected builds?

ОПИСАНИЕ

Tools (nx affected, turborepo) определяют зависимый граф packages, билдят/тестируют только affected. CI fast feedback. Base-branch comparison.

КАК РАБОТАЕТ

- 01 Dep-graph analysis.
- 02 Affected = changed + dependents.
- 03 nx affected:build / test.
- 04 Compare с base-branch (main).

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Large monorepos.

ПОДВОДНЫЕ КАМНИ

- 01 Wrong base-branch → re-build everything.
- 02 Cache misses – slow.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое graph analysis?
- 02 Чем remote cache полезен?

ПОЛНАЯ СТАТЬЯ

<https://nx.dev/nx-api/nx/documents/affected>

В О П Р О С Q 3 4

Как версионировать shared packages?

ОПИСАНИЕ

Changesets (changeset-bot создаёт PR с version-bumps). Semantic-release. Independent vs lockstep versioning. Per-package CHANGELOG. Auto-publish через CI.

КАК РАБОТАЕТ

- 01 Changesets bot.
- 02 Semantic-release.
- 03 Independent vs lockstep.
- 04 Auto-publish CI.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Public npm libs.
- 02 Internal monorepo packages.

ПОДВОДНЫЕ КАМНИ

- 01 Manual versioning – drift.
- 02 Lockstep при independent updates – wasteful versions.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое changesets?
- 02 Чем lockstep от independent отличается?

ПОЛНАЯ СТАТЬЯ

<https://github.com/changesets/changesets>

Когда лучше modular monolith, а не microfrontends?

ОПИСАНИЕ

Single team, single repo, simpler deploy. Когда coordination overhead MF превышает benefits. Можно начать modular monolith и при росте перейти к MF.

КАК РАБОТАЕТ

- 01 Single team / coordinated releases.
- 02 Less complexity.
- 03 Easier debug / tooling.
- 04 Migration path к MF возможна.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Small-medium teams.
- 02 Early-stage products.

ПОДВОДНЫЕ КАМНИ

- 01 MF adopted без need – premature.
- 02 Modular monolith forever без boundaries.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое modular monolith?
- 02 Когда переходить к MF?

ПОЛНАЯ СТАТЬЯ

<https://martinfowler.com/articles/2024/monolith-decomposition-pattern.html>

ФИНАЛЬНЫЙ ЧЕК - ЛИСТ

СДАЛ ЛИ ЗАЧЁТ

ПРОЙДИ ВСЛУХ БЕЗ ПОДГЛЯДЫВАНИЙ ЕСЛИ ЗАСТРЯЛ – ВЕРНИСЬ В НУЖНЫЙ БИЛЕТ

- [] Описать MFE / когда / overengineering / Module Federation (Q1-Q4)
- [] Описать host / remote / shared / risks / version skew (Q5-Q9)
- [] Описать React shared / independent deploy / runtime compat / contracts (Q10-Q13)
- [] Шарить types / DS / auth / routing (Q14-Q17)
- [] Описать error isolation / fallback / monitoring / rollback (Q18-Q21)
- [] Описать testing / local dev / UX / migration / strangler (Q22-Q27)
- [] Сравнить monorepo / MF, описать packages, configs, affected, versioning, modular monolith (Q28-Q35)

EVENT LOOP / ASYNC МЕТОДИЧКА v1

МАТЕРИАЛ ОПИРАЕТСЯ НА learn.javascript.ru